UD3.1: List Comprehensions

Módulo: ECP — Estructuras de Control en Python (5099)
Unidad: UD3 — Programación Avanzada

Índice de contenidos

1. Motivación: un patrón que se repite
2. Sintaxis de list comprehension
3. Comprehensions de transformación
4. Comprehensions con filtro (if)
5. Transformación y filtro combinados
6. Expresión condicional (if/else)
7. Usando funciones en comprehensions
8. Cuándo usar y cuándo NO usar comprehensions
9. Errores comunes
10. Ejercicios de clase
11. Ejercicios para casa

1. Motivación: un patrón que se repite

En la programación con bucles `for` es habitual repetir ciertos patrones:

Patrón 1 — Crear una lista transformando cada elemento:

```
cuadrados = []
for i in range(1, 11):
    cuadrados.append(i ** 2)
```

Patrón 2 — Crear una lista filtrando elementos:

```
pares = []
for i in range(1, 21):
    if i % 2 == 0:
        pares.append(i)
```

Patrón 3 — Transformar y filtrar a la vez:

```
cuadrados_pares = []
for i in range(1, 21):
    if i % 2 == 0:
        cuadrados_pares.append(i ** 2)
```

Los tres patrones comparten la misma estructura:

1. Crear una lista vacía
2. Recorrer una secuencia con `for`
3. (Opcionalmente) comprobar una condición con `if`
4. Añadir un valor con `.append()`

Este patrón es tan frecuente que Python ofrece una sintaxis compacta para expresarlo: las **list comprehensions** (comprensiones de listas).

2. Sintaxis de list comprehension

Forma básica (transformación)

```
nueva_lista = [expresión for variable in iterable]
```

Equivale a:

```
nueva_lista = []
for variable in iterable:
    nueva_lista.append(expresión)
```

Forma con filtro

```
nueva_lista = [expresión for variable in iterable if condición]
```

Equivale a:

```
nueva_lista = []
for variable in iterable:
    if condición:
        nueva_lista.append(expresión)
```

Cómo leerla

Una comprehension se lee de izquierda a derecha. El orden lógico es:

```
[expresión  for variable in iterable  if condición]
 _________  __________________________  ___________

  ¿qué        ¿de dónde viene          ¿cuáles
  quiero?      cada elemento?           me quedo?
```

Ejemplo: `[i ** 2 for i in range(1, 11) if i % 2 == 0]`

Lectura: *"el cuadrado de* `i` *, para cada* `i` *del 1 al 10, si* `i` *es par"*.

3. Comprehensions de transformación

Son comprehensions **sin** `if` : toman cada elemento y lo transforman.

Ejemplo 1: Números del 1 al 10

```
# Con for
numeros = []
for i in range(1, 11):
    numeros.append(i)

# Con comprehension
numeros = [i for i in range(1, 11)]

# Resultado: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

> **Nota**: en este caso concreto, `list(range(1, 11))` sería más directo. Las comprehensions son especialmente útiles cuando hay una **transformación** (la expresión no es simplemente `i`).

Ejemplo 2: Cuadrados

```
# Con for
cuadrados = []
for i in range(1, 11):
    cuadrados.append(i ** 2)

# Con comprehension
cuadrados = [i ** 2 for i in range(1, 11)]

# Resultado: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Ejemplo 3: Tabla de multiplicar del 7

```
# Con for
tabla = []
for i in range(1, 11):
    tabla.append(f"7 x {i} = {7 * i}")

# Con comprehension
tabla = [f"7 x {i} = {7 * i}" for i in range(1, 11)]

# Resultado: ['7 x 1 = 7', '7 x 2 = 14', ..., '7 x 10 = 70']
```

Ejemplo 4: Longitudes de palabras

```python
palabras = ["hola", "mundo", "python", "es", "genial"]

# Con for
longitudes = []
for palabra in palabras:
    longitudes.append(len(palabra))

# Con comprehension
longitudes = [len(palabra) for palabra in palabras]

# Resultado: [4, 5, 6, 2, 6]
```

Ejemplo 5: Convertir a mayúsculas

```python
nombres = ["ana", "carlos", "maría", "juan"]

# Con for
mayusculas = []
for nombre in nombres:
    mayusculas.append(nombre.upper())

# Con comprehension
mayusculas = [nombre.upper() for nombre in nombres]

# Resultado: ['ANA', 'CARLOS', 'MARÍA', 'JUAN']
```

Ejemplo 6: Convertir temperaturas de Celsius a Fahrenheit

```
celsius = [0, 10, 20, 30, 40]

# Con for
fahrenheit = []
for c in celsius:
    fahrenheit.append(c * 9/5 + 32)

# Con comprehension
fahrenheit = [c * 9/5 + 32 for c in celsius]

# Resultado: [32.0, 50.0, 68.0, 86.0, 104.0]
```

Ejemplo 7: Doble de cada nota

```
notas = [3, 7, 8, 4, 6]

# Con for
dobles = []
for nota in notas:
    dobles.append(nota * 2)

# Con comprehension
dobles = [nota * 2 for nota in notas]

# Resultado: [6, 14, 16, 8, 12]
```

4. Comprehensions con filtro (if)

La cláusula `if` permite seleccionar solo los elementos que cumplen una condición.

Ejemplo 8: Números pares del 1 al 20

```python
# Con for + if (Ejercicio 4 de UD1)
pares = []
for i in range(1, 21):
    if i % 2 == 0:
        pares.append(i)

# Con comprehension
pares = [i for i in range(1, 21) if i % 2 == 0]

# Resultado: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

Ejemplo 9: Múltiplos de 3 entre 1 y 50

```python
# Con for + if (Ejercicio 8 de UD1)
multiplos = []
for i in range(1, 51):
    if i % 3 == 0:
        multiplos.append(i)

# Con comprehension
multiplos = [i for i in range(1, 51) if i % 3 == 0]

# Para contar cuántos hay:
print(len(multiplos))  # 16
```

Ejemplo 10: Filtrar notas aprobadas

```python
notas = [3, 7, 8, 2, 6, 9, 4, 5, 1, 10]

# Con for + if
aprobados = []
for nota in notas:
    if nota >= 5:
        aprobados.append(nota)

# Con comprehension
aprobados = [nota for nota in notas if nota >= 5]

# Resultado: [7, 8, 6, 9, 5, 10]
```

Ejemplo 11: Palabras largas (más de 4 letras)

```python
palabras = ["sol", "python", "hoy", "programar", "if", "variable"]

# Con for + if
largas = []
for p in palabras:
    if len(p) > 4:
        largas.append(p)

# Con comprehension
largas = [p for p in palabras if len(p) > 4]

# Resultado: ['python', 'programar', 'variable']
```

Ejemplo 12: Números positivos de una lista

```
datos = [5, -3, 8, -1, 0, 4, -2, 7]

# Con for + if
positivos = []
for num in datos:
    if num > 0:
        positivos.append(num)

# Con comprehension
positivos = [num for num in datos if num > 0]

# Resultado: [5, 8, 4, 7]
```

Ejemplo 13: Vocales de una cadena

```
texto = "Hola mundo"

# Con for + if
vocales = []
for letra in texto:
    if letra.lower() in "aeiou":
        vocales.append(letra)

# Con comprehension
vocales = [letra for letra in texto if letra.lower() in "aeiou"]

# Resultado: ['o', 'a', 'u', 'o']
```

5. Transformación y filtro combinados

La combinación más potente: seleccionar elementos **y** transformarlos simultáneamente.

Ejemplo 14: Cuadrados de los pares

```python
# Con for + if
cuadrados_pares = []
for i in range(1, 11):
    if i % 2 == 0:
        cuadrados_pares.append(i ** 2)

# Con comprehension
cuadrados_pares = [i ** 2 for i in range(1, 11) if i % 2 == 0]

# Resultado: [4, 16, 36, 64, 100]
```

Ejemplo 15: Notas aprobadas con bonus de +1

```python
notas = [3, 7, 8, 2, 6, 9, 4, 5, 1, 10]

# Con for + if
aprobados_bonus = []
for nota in notas:
    if nota >= 5:
        aprobados_bonus.append(nota + 1)

# Con comprehension
aprobados_bonus = [nota + 1 for nota in notas if nota >= 5]

# Resultado: [8, 9, 7, 10, 6, 11]
```

Ejemplo 16: Iniciales de nombres largos

```
nombres = ["Ana", "Alejandro", "Bo", "Cristina", "Li", "Francisco"]

# Con for + if
iniciales = []
for nombre in nombres:
    if len(nombre) > 3:
        iniciales.append(nombre[0])

# Con comprehension
iniciales = [nombre[0] for nombre in nombres if len(nombre) > 3]

# Resultado: ['A', 'C', 'F']
```

Ejemplo 17: Palabras en mayúsculas que contienen la letra 'a'

```
palabras = ["casa", "sol", "mesa", "luz", "playa", "bar"]

# Con for + if
resultado = []
for p in palabras:
    if "a" in p:
        resultado.append(p.upper())

# Con comprehension
resultado = [p.upper() for p in palabras if "a" in p]

# Resultado: ['CASA', 'MESA', 'PLAYA', 'BAR']
```

6. Expresión condicional (if/else)

En las secciones anteriores, el `if` se usa al **final** de la comprehension para **filtrar** elementos (la lista resultante tiene menos elementos que la original). Existe otra forma de usar `if` que no filtra, sino que **transforma** cada elemento de forma diferente según una condición.

Diferencia clave

Uso	Sintaxis	Efecto
Filtro	`[expr for x in seq **if** cond]`	Descarta elementos que no cumplen la condición. La lista puede ser más corta.
Transformación condicional	`[expr1 **if** cond **else** expr2 for x in seq]`	Transforma **todos** los elementos, pero de forma distinta según la condición. La lista tiene la misma longitud.

IMPORTANTE: La posición del `if` determina su comportamiento:

- `if` **después** del `for` → **filtro** (puede ir sin `else`)
- `if` **antes** del `for` → **expresión condicional** (requiere `else` obligatoriamente)

Ejemplo 18: Clasificar números como positivo/negativo

```
numeros = [3, -1, 4, -5, 2, -8]

# Con for
clasificados = []
for n in numeros:
    if n >= 0:
        clasificados.append("positivo")
    else:
        clasificados.append("negativo")

# Con comprehension (if/else como transformación)
clasificados = ["positivo" if n >= 0 else "negativo" for n in numeros]

# Resultado: ['positivo', 'negativo', 'positivo', 'negativo', 'positivo', 'negativo']
# Nota: la lista tiene 6 elementos, igual que la original (no se filtra nada)
```

Ejemplo 19: Aprobar o suspender

```
notas = [3, 7, 4, 9, 5, 2, 8]

# Con for
resultados = []
for nota in notas:
    if nota >= 5:
        resultados.append("Aprobado")
    else:
        resultados.append("Suspenso")

# Con comprehension
resultados = ["Aprobado" if nota >= 5 else "Suspenso" for nota in notas]

# Resultado: ['Suspenso', 'Aprobado', 'Suspenso', 'Aprobado', 'Aprobado', 'Suspenso', 'Aprobado
```

Ejemplo 20: Limitar valores a un máximo

```
notas = [7, 12, 5, 15, 9, 3]

# Limitar a 10 como máximo
limitadas = [min(nota, 10) for nota in notas]
# Resultado: [7, 10, 5, 10, 9, 3]

# Equivalente con if/else
limitadas = [10 if nota > 10 else nota for nota in notas]
# Resultado: [7, 10, 5, 10, 9, 3]
```

Ejemplo 21: Reemplazar valores negativos por cero

```
datos = [5, -3, 8, -1, 0, 4, -2]

# Con for
limpios = []
for x in datos:
    if x < 0:
        limpios.append(0)
    else:
        limpios.append(x)

# Con comprehension
limpios = [0 if x < 0 else x for x in datos]

# Resultado: [5, 0, 8, 0, 0, 4, 0]
```

Comparación directa: filtro vs transformación condicional

```
notas = [3, 7, 4, 9, 5, 2, 8]

# FILTRO: solo las aprobadas (descarta las demás)
aprobadas = [n for n in notas if n >= 5]
# Resultado: [7, 9, 5, 8]  ← 4 elementos (se han descartado 3)

# TRANSFORMACIÓN: todas, pero con etiqueta
etiquetas = ["AP" if n >= 5 else "SU" for n in notas]
# Resultado: ['SU', 'AP', 'SU', 'AP', 'AP', 'SU', 'AP']  ← 7 elementos (todos)
```

Se pueden combinar filtro y transformación condicional

```
numeros = [-5, 3, -2, 8, 0, -1, 7, 4]

# Solo los distintos de cero, reemplazando negativos por su valor absoluto
resultado = [abs(n) if n < 0 else n for n in numeros if n != 0]
# Resultado: [5, 3, 2, 8, 1, 7, 4]
```

En este caso:

- `if n != 0` (al final) → **filtra**: descarta el 0

- `abs(n) if n < 0 else n` (al principio) → **transforma**: convierte negativos a positivos

> **Precaución**: combinar ambas formas en una sola comprehension puede dificultar la lectura. Si el resultado no es claro a primera vista, es preferible usar un bucle `for` convencional.

7. Usando funciones en comprehensions

Las comprehensions son compatibles con cualquier tipo de función: tanto funciones built-in de Python como funciones definidas por el usuario.

Con funciones de Python (built-in)

```
numeros = ["-3", "7", "12", "-5", "0", "8"]

# Convertir strings a enteros
enteros = [int(x) for x in numeros]
# Resultado: [-3, 7, 12, -5, 0, 8]

# Valores absolutos
absolutos = [abs(int(x)) for x in numeros]
# Resultado: [3, 7, 12, 5, 0, 8]
```

Con funciones propias

```python
def es_aprobado(nota):
    return nota >= 5

def redondear_nota(nota):
    if nota >= 4.5:
        return round(nota)
    return int(nota)

notas = [3.2, 7.8, 4.6, 8.1, 2.9, 5.5, 6.3]

# Filtrar con función propia
aprobados = [nota for nota in notas if es_aprobado(nota)]
# Resultado: [7.8, 5.5, 6.3]

# Transformar con función propia
redondeadas = [redondear_nota(nota) for nota in notas]
# Resultado: [3, 8, 5, 8, 2, 6, 6]

# Combinado: redondear solo las aprobadas
aprobados_redondeados = [redondear_nota(nota) for nota in notas if es_aprobado(nota)]
# Resultado: [8, 6, 6]
```

Con métodos de string

```python
nombres = ["  ana ", " CARLOS", "María  ", "juan"]

# Limpiar y capitalizar
limpios = [nombre.strip().capitalize() for nombre in nombres]
# Resultado: ['Ana', 'Carlos', 'María', 'Juan']
```

8. Cuándo usar y cuándo NO usar comprehensions

Usar comprehension cuando...

Situación	Ejemplo
Transformar cada elemento	`[x * 2 for x in lista]`
Filtrar elementos	`[x for x in lista if x > 0]`
Transformar + filtrar	`[x.upper() for x in palabras if len(x) > 3]`
Convertir tipos	`[int(x) for x in strings]`
Aplicar una función a cada elemento	`[len(p) for p in palabras]`

NO usar comprehension cuando...

Situación	Por qué no	Alternativa
La lógica es compleja	Se vuelve ilegible	Usar un `for` normal
Necesitas `break` o `continue`	No se pueden usar	Usar un `for` normal
Necesitas `print()` o efectos secundarios	Las comprehensions son para **crear listas**, no para ejecutar acciones	Usar un `for` normal
Necesitas acumular un único valor	No estás creando una lista	Usar `sum()`, `max()`, `min()` o un `for`
Necesitas varias condiciones complejas	Difícil de leer	Usar un `for` o extraer a función

Ejemplo de comprehension que NO deberías escribir

```
# ❌ MAL - Demasiado complejo
resultado = [x ** 2 if x > 0 else -x for x in numeros if x != 0 and x % 2 == 0]

# ✅ BIEN - Usa un for cuando es complejo
resultado = []
for x in numeros:
    if x != 0 and x % 2 == 0:
        if x > 0:
            resultado.append(x ** 2)
        else:
            resultado.append(-x)
```

Ejemplo de uso INCORRECTO de comprehension

```
# ❌ MAL - Usando comprehension para ejecutar acciones (print)
[print(x) for x in lista]  # "Funciona" pero crea una lista de None inútil

# ✅ BIEN - Usar for para acciones
for x in lista:
    print(x)
```

Regla práctica: si la comprehension no cabe en una línea (~80 caracteres) o es necesario leerla dos veces para entenderla, es preferible usar un `for` .

9. Errores comunes

❌ Olvidar los corchetes

```
# ❌ Error de sintaxis
resultado = x ** 2 for x in range(10)

# ✅ Correcto
resultado = [x ** 2 for x in range(10)]
```

❌ Poner el if antes del for

```
# ❌ Error de sintaxis
resultado = [x if x > 0 for x in numeros]

# ✅ Correcto (filtro)
resultado = [x for x in numeros if x > 0]
```

IMPORTANTE: la forma `[x if condición else y for ...]` existe pero hace algo distinto: es una **expresión condicional** (ternario), no un filtro. Ver sección 6.

❌ Confundir iterar valores con iterar índices

```
notas = [7, 4, 9, 5, 8]

# ❌ Iterar sobre índices innecesariamente
dobles = [notas[i] * 2 for i in range(len(notas))]

# ✅ Iterar directamente sobre valores
dobles = [nota * 2 for nota in notas]
```

❌ Modificar la lista original en vez de crear una nueva

```
notas = [7, 4, 9, 5, 8]

# ❌ Esto NO modifica notas
[nota + 1 for nota in notas]  # Se crea una lista y se descarta

# ✅ Hay que asignar el resultado
notas_subidas = [nota + 1 for nota in notas]
```

❌ Usar append dentro de una comprehension

```
resultado = []

# ❌ Mezclar comprehension con append
[resultado.append(x ** 2) for x in range(10)]  # Funciona pero es horrible

# ✅ Elegir UNO de los dos estilos
resultado = [x ** 2 for x in range(10)]  # Comprehension

# O bien:
resultado = []
for x in range(10):
    resultado.append(x ** 2)  # For loop
```

10. Ejercicios de clase

Convierte cada bloque de código con `for` a una list comprehension equivalente.

Ejercicio 1: Cubos del 1 al 10

```
cubos = []
for i in range(1, 11):
    cubos.append(i ** 3)
```

Ejercicio 2: Primeras letras

```
nombres = ["Ana", "Borja", "Carmen", "David"]
iniciales = []
for nombre in nombres:
    iniciales.append(nombre[0])
```

Ejercicio 3: Solo negativos

```
numeros = [4, -1, 7, -3, 0, -8, 2]
negativos = []
for n in numeros:
    if n < 0:
        negativos.append(n)
```

Ejercicio 4: Longitud de palabras largas

```
palabras = ["a", "hola", "sol", "programar", "si", "python"]
longitudes = []
for p in palabras:
    if len(p) > 3:
        longitudes.append(len(p))
```

Ejercicio 5: Precios con IVA (solo los mayores de 10€)

```
precios = [5, 15, 8, 25, 3, 12, 50]
con_iva = []
for precio in precios:
    if precio > 10:
        con_iva.append(round(precio * 1.21, 2))
```

11. Ejercicios para casa

Bloque A — Conversiones directas

Convierte cada programa a una list comprehension equivalente. Comprueba que el resultado es el mismo.

A1. Tabla de multiplicar del 9

```python
tabla = []
for i in range(1, 11):
    tabla.append(f"9 x {i} = {9 * i}")
print(tabla)
# Resultado esperado: ['9 x 1 = 9', '9 x 2 = 18', ..., '9 x 10 = 90']
```

A2. Números al revés

```python
numeros = [1, 2, 3, 4, 5]
reversos = []
for n in numeros:
    reversos.append(-n)
print(reversos)
# Resultado esperado: [-1, -2, -3, -4, -5]
```

A3. Divisibles por 5 entre 1 y 100

```python
divisibles = []
for i in range(1, 101):
    if i % 5 == 0:
        divisibles.append(i)
print(divisibles)
# Resultado esperado: [5, 10, 15, ..., 100]
```

A4. Palabras que empiezan por vocal

```python
palabras = ["arbol", "casa", "elefante", "mesa", "isla", "perro", "uva"]
con_vocal = []
for p in palabras:
    if p[0] in "aeiou":
        con_vocal.append(p)
print(con_vocal)
# Resultado esperado: ['arbol', 'elefante', 'isla', 'uva']
```

A5. Notas subidas un punto (solo las suspensas)

```
notas = [3, 7, 4, 9, 2, 6, 5, 1]
recuperadas = []
for nota in notas:
    if nota < 5:
        recuperadas.append(nota + 1)
print(recuperadas)
# Resultado esperado: [4, 5, 3, 2]
```

Bloque B — Escribir comprehensions desde cero

Escribe una list comprehension (sin escribir primero el for) que produzca el resultado indicado.

B1. Mitades

Dada `numeros = [2, 7, 10, 15, 20, 3]`, crea una lista con la mitad de cada número.

```
Resultado esperado: [1.0, 3.5, 5.0, 7.5, 10.0, 1.5]
```

B2. Solo strings

Dada `mezcla = [1, "hola", 3.14, "mundo", True, "python"]`, crea una lista que contenga solo los elementos de tipo `str`.

```
Pista: usa isinstance(x, str) o type(x) == str
Resultado esperado: ['hola', 'mundo', 'python']
```

B3. Índices de aprobados

Dada `notas = [3, 7, 4, 9, 2, 6, 5, 1]`, crea una lista con los **índices** (posiciones) de las notas aprobadas (>= 5).

```
Pista: necesitarás range(len(notas)) para tener acceso al índice
Resultado esperado: [1, 3, 5, 6]
```

B4. Palabras sin espacios

Dada `frase = "  hola   mundo  cruel  "`, crea una lista con cada palabra de la frase (sin espacios vacíos).

```
Pista: split() separa por espacios, pero puede dejar strings vacíos. ¿O no?
Resultado esperado: ['hola', 'mundo', 'cruel']
```

B5. Números de Fibonacci menores que 100

Dada la lista de Fibonacci `fib = [1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233]`, crea una lista solo con los que sean menores que 100.

```
Resultado esperado: [1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

Bloque C — Desafíos

C1. Producto cartesiano

Dadas dos listas `colores = ["rojo", "azul"]` y `tallas = ["S", "M", "L"]`, crea una lista con todas las combinaciones posibles como tuplas.

```
Resultado esperado: [('rojo', 'S'), ('rojo', 'M'), ('rojo', 'L'), ('azul', 'S'), ('azul', 'M'),
Pista: se pueden anidar dos for dentro de una comprehension
```

C2. Aplanar una lista de listas

Dada `matriz = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]`, crea una lista plana con todos los elementos.

```
Resultado esperado: [1, 2, 3, 4, 5, 6, 7, 8, 9]
Pista: dos for anidados, el primero recorre filas, el segundo recorre elementos
```

C3. Fizzbuzz con comprehension

Crea una lista del 1 al 20 donde:

- Si el número es divisible por 3 y por 5, aparece `"FizzBuzz"`
- Si es divisible solo por 3, aparece `"Fizz"`
- Si es divisible solo por 5, aparece `"Buzz"`
- En otro caso, aparece el número como string

```
Resultado esperado: ['1', '2', 'Fizz', '4', 'Buzz', 'Fizz', '7', '8', 'Fizz', 'Buzz', '11', 'Fi
Pista: necesitarás expresiones condicionales (ternarios) anidadas. Si te resulta ilegible, eso
```

Tabla resumen de ejercicios

N.º	Enunciado	Tipo	Dificultad
A1	Tabla de multiplicar del 9	Conversión (transformación)	Básica
A2	Números al revés	Conversión (transformación)	Básica
A3	Divisibles por 5	Conversión (filtro)	Básica
A4	Palabras que empiezan por vocal	Conversión (filtro)	Media
A5	Notas suspensas +1	Conversión (filtro + transformación)	Media
B1	Mitades	Crear (transformación)	Básica
B2	Solo strings	Crear (filtro con tipo)	Media
B3	Índices de aprobados	Crear (filtro con índice)	Media
B4	Palabras sin espacios	Crear (split + filtro)	Media
B5	Fibonacci < 100	Crear (filtro)	Básica
C1	Producto cartesiano	Desafío (for anidados)	Alta
C2	Aplanar matriz	Desafío (for anidados)	Alta
C3	FizzBuzz	Desafío (ternarios)	Muy Alta

Resumen

Patrón con `for`	Comprehension equivalente	Tipo
`for x in seq: res.append(x)`	`[x for x in seq]`	Copia
`for x in seq: res.append(f(x))`	`[f(x) for x in seq]`	Transformación
`for x in seq: + if cond: + res.append(x)`	`[x for x in seq if cond]`	Filtro

Patrón con `for`	Comprehension equivalente	Tipo
`for x in seq:` + `if cond:` + `res.append(f(x))`	`[f(x) for x in seq if cond]`	Filtro + transformación
`for x in seq:` + `if cond:` + `res.append(a)` + `else:` + `res.append(b)`	`[a if cond else b for x in seq]`	Transformación condicional

Reglas de oro:

- Si el objetivo es crear una lista nueva → comprehension es buena opción
- Si la lógica es simple (una transformación, un filtro) → comprehension
- Si la lógica es compleja, requiere `break` / `continue`, o no se crea una lista → usar `for`
- Si no se entiende en una lectura → demasiado compleja, usar `for`

Recursos adicionales

- **Documentación Python** — List Comprehensions
- **Python Tutor** — pythontutor.com (visualiza la ejecución paso a paso)
- **Real Python** — When to Use a List Comprehension in Python